



Transformers documentation

Quickstart ▾



Join the Hugging Face community

and get access to the augmented documentation experience

[Sign Up](#)

to get started



Quickstart

[Open in Colab](#)[Open](#)[Studio Lab](#)[Copy page](#)

Transformers is designed to be fast and easy to use so that everyone can start learning or building with transformer models.

The number of user-facing abstractions is limited to only three classes for instantiating a model, and two APIs for inference or training. This quickstart introduces you to Transformers' key features and shows you how to:

- load a pretrained model
- run inference with [Pipeline](#)
- fine-tune a model with [Trainer](#)

Set up

To start, we recommend creating a Hugging Face [account](#). An account lets you host and access version controlled models, datasets, and [Spaces](#) on the Hugging Face [Hub](#), a collaborative platform for discovery and building.

Create a [User Access Token](#) and log in to your account.

[Ask HuggingChat](#)[notebook](#)[CLI](#)

Paste your User Access Token into [notebook_login](#) when prompted to log in.

```
from huggingface_hub import notebook_login

notebook_login()
```

Install PyTorch.

```
!pip install torch
```

Then install an up-to-date version of Transformers and some additional libraries from the Hugging Face ecosystem for accessing datasets and vision models, evaluating training, and optimizing training for large models.

```
!pip install -U transformers datasets evaluate accelerate timm
```

Pretrained models

Each pretrained model inherits from three base classes.

Class	Description
PreTrainedConfig	A file that specifies a models attributes such as the number of attention heads or vocabulary size.
PreTrainedModel	A model (or architecture) defined by the model attributes from the configuration file. A pretrained model only returns the raw hidden states. For a specific task, use the appropriate model head to convert the raw hidden states into a meaningful result (for example, LlamaModel versus LlamaForCausalLM).
Preprocessor	A class for converting raw inputs (text, images, audio, multimodal) into numerical inputs to the model. For example, PreTrainedTokenizer converts text into tensors and ImageProcessingMixin converts pixels into tensors.



We recommend using the [AutoClass](#) API to load models and preprocessors because it automatically infers the appropriate architecture for each task and machine learning framework based on the name or path to the pretrained weights and configuration file.

Use [from_pretrained\(\)](#) to load the weights and configuration file from the Hub into the model and preprocessor class.

When you load a model, configure the following parameters to ensure the model is optimally loaded.

- `device_map="auto"` automatically allocates the model weights to your fastest device first.
- `dtype="auto"` directly initializes the model weights in the data type they're stored in, which can help avoid loading the weights twice (PyTorch loads weights in `torch.float32` by default).

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-hf", dtype="auto", de
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b-hf")
```

Tokenize the text and return PyTorch tensors with the tokenizer. Move the model to an accelerator if it's available to accelerate inference.

```
model_inputs = tokenizer(["The secret to baking a good cake is "], return_tensors="pt").t
```

The model is now ready for inference or training.

For inference, pass the tokenized inputs to [generate\(\)](#) to generate text. Decode the token ids back into text with [batch_decode\(\)](#).

```
generated_ids = model.generate(**model_inputs, max_length=30)
tokenizer.batch_decode(generated_ids)[0]
```



```
'<s> The secret to baking a good cake is 100% in the preparation. There are so many recip
```

Skip ahead to the [Trainer](#) section to learn how to fine-tune a model.

Pipeline

The [Pipeline](#) class is the most convenient way to inference with a pretrained model. It supports many tasks such as text generation, image segmentation, automatic speech recognition, document question answering, and more.

Refer to the [Pipeline](#) API reference for a complete list of available tasks.

Create a [Pipeline](#) object and select a task. By default, [Pipeline](#) downloads and caches a default pretrained model for a given task. Pass the model name to the `model` parameter to choose a specific model.

text generation

image segmentation

automatic speech recognition

Use `Accelerator` to automatically detect an available accelerator for inference.

```
from transformers import pipeline
from accelerate import Accelerator

device = Accelerator().device

pipeline = pipeline("text-generation", model="meta-llama/Llama-2-7b-hf", device=device)
```

Prompt [Pipeline](#) with some initial text to generate more text.

```
pipeline("The secret to baking a good cake is ", max_length=50)
[{'generated_text': 'The secret to baking a good
```

Trainer

Trainer is a complete training and evaluation loop for PyTorch models. It abstracts away a lot of the boilerplate usually involved in manually writing a training loop, so you can start training faster and focus on training design choices. You only need a model, dataset, a preprocessor, and a data collator to build batches of data from the dataset.

Use the TrainingArguments class to customize the training process. It provides many options for training, evaluation, and more. Experiment with training hyperparameters and features like batch size, learning rate, mixed precision, torch.compile, and more to meet your training needs. You could also use the default training parameters to quickly produce a baseline.

Load a model, tokenizer, and dataset for training.

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset

model = AutoModelForSequenceClassification.from_pretrained("distilbert/distilbert-base-ur
tokenizer = AutoTokenizer.from_pretrained("distilbert/distilbert-base-uncased")
dataset = load_dataset("rotten_tomatoes")
```

Create a function to tokenize the text and convert it into PyTorch tensors. Apply this function to the whole dataset with the map method.

```
def tokenize_dataset(dataset):
    return tokenizer(dataset["text"])
dataset = dataset.map(tokenize_dataset, batched=True)
```

Load a data collator to create batches of data and pass the tokenizer to it.

```
from transformers import DataCollatorWithPadding

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

Next, set up TrainingArguments with the training features and hyperparameters.

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="distilbert-rotten-tomatoes",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    push_to_hub=True,
)
```

Finally, pass all these separate components to [Trainer](#) and call [train\(\)](#) to start.

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
)

trainer.train()
```

Share your model and tokenizer to the Hub with [push_to_hub\(\)](#).

```
trainer.push_to_hub()
```

Congratulations, you just trained your first model with Transformers!

Next steps

Now that you have a better understanding of Transformers and what it offers, it's time to keep exploring and learning what interests you the most.



- **Base classes:** Learn more about the configuration, model and processor classes. This will help you understand how to create and customize models, preprocess different types of inputs (audio, images, multimodal), and how to share your model.
- **Inference:** Explore the [Pipeline](#) further, inference and chatting with LLMs, agents, and how to optimize inference with your machine learning framework and hardware.
- **Training:** Study the [Trainer](#) in more detail, as well as distributed training and optimizing training on specific hardware.
- **Quantization:** Reduce memory and storage requirements with quantization and speed up inference by representing weights with fewer bits.
- **Resources:** Looking for end-to-end recipes for how to train and inference with a model for a specific task? Check out the task recipes!

[Update](#) on GitHub

← Installation

Dynamic weight loading →





Transformers documentation

Installation ▾



Join the Hugging Face community

and get access to the augmented documentation experience

[Sign Up](#)

to get started



Installation

[Copy page](#)

Transformers works with [PyTorch](#). It has been tested on Python 3.10+ and PyTorch 2.4+.

Virtual environment

[uv](#) is an extremely fast Rust-based Python package and project manager and requires a [virtual environment](#) by default to manage different projects and avoids compatibility issues between dependencies.

It can be used as a drop-in replacement for [pip](#), but if you prefer to use pip, remove uv from the commands below.

Refer to the [uv installation](#) docs to install uv.

Create a virtual environment to install Transformers in.

```
uv venv .env
source .env/bin/activate
```

[Ask HuggingChat](#)

Python

Install Transformers with the following command.

uv is a fast Rust-based Python package and project manager.

```
uv pip install transformers
```

For GPU acceleration, install the appropriate CUDA drivers for PyTorch.

Run the command below to check if your system detects an NVIDIA GPU.

```
nvidia-smi
```

To install a CPU-only version of Transformers, run the following command.

```
uv pip install torch --index-url https://download.pytorch.org/whl/cpu
uv pip install transformers
```

Test whether the install was successful with the following command. It should return a label and score for the provided text.

```
python -c "from transformers import pipeline; print(pipeline('sentiment-analysis')('huggi
[{'label': 'POSITIVE', 'score': 0.9998704791069031}])"
```

Source install

Installing from source installs the *latest* version rather than the *stable* version of the library. It ensures you have the most up-to-date changes in Transformers and it's useful for experimenting with the latest features or fixing a bug that hasn't been officially released in the stable version yet.

The downside is that the latest version may not always be stable. If you encounter any problems please open a [GitHub Issue](#) so we can fix it as soon as possible.

Install from source with the following command.

```
uv pip install git+https://github.com/huggingface/transformers
```

Check if the install was successful with the command below. It should return a label and score for the provided text.

```
python -c "from transformers import pipeline; print(pipeline('sentiment-analysis')('huggi  
[{'label': 'POSITIVE', 'score': 0.9998704791069031}])"
```

Editable install

An editable install is useful if you're developing locally with Transformers. It links your local copy of Transformers to the Transformers repository instead of copying the files. The files are added to Python's import path.

```
git clone https://github.com/huggingface/transformers.git  
cd transformers  
uv pip install -e .
```

You must keep the local Transformers folder to keep using it.

Update your local version of Transformers with the latest changes in the main repository with the following command.

```
cd ~/transformers/  
git pull
```

conda

conda is a language-agnostic package manager. Install it in your newly created virtual environment.



```
conda install conda-forge::transformers
```

Set up

After installation, you can configure the Transformers cache location or set up the library for offline usage.

Cache directory

When you load a pretrained model with `from_pretrained()`, the model is downloaded from the Hub and locally cached.

Every time you load a model, it checks whether the cached model is up-to-date. If it's the same, then the local model is loaded. If it's not the same, the newer model is downloaded and cached.

The default directory given by the shell environment variable `HF_HUB_CACHE` is

`~/.cache/huggingface/hub`. On Windows, the default directory is `C:\Users\username\.cache\huggingface\hub`.

Cache a model in a different directory by changing the path in the following shell environment variables (listed by priority).

1. `HF_HUB_CACHE` (default)
2. `HF_HOME`
3. `XDG_CACHE_HOME` + `/huggingface` (only if `HF_HOME` is not set)

Offline mode

To use Transformers in an offline or firewalled environment requires the downloaded and cached files ahead of time. Download a model repository from the Hub with the `snapshot_download` method.



Refer to the [Download files from the Hub](#) guide for more options for downloading files from the Hub. You can download files from specific revisions, download from the CLI, and even filter which files to download from a repository.

```
from huggingface_hub import snapshot_download

snapshot_download(repo_id="meta-llama/Llama-2-7b-hf", repo_type="model")
```

Set the environment variable `HF_HUB_OFFLINE=1` to prevent HTTP calls to the Hub when loading a model.

```
HF_HUB_OFFLINE=1 \
python examples/pytorch/language-modeling/run_clm.py --model_name_or_path meta-llama/Llan
```

Another option for only loading cached files is to set `local_files_only=True` in [from_pretrained\(\)](#).

```
from transformers import LlamaForCausalLM

model = LlamaForCausalLM.from_pretrained("./path/to/local/directory", local_files_only=True)
```

[Update](#) on GitHub

← Transformers

Quickstart →





Transformers documentation

Transformers ▾



Join the Hugging Face community

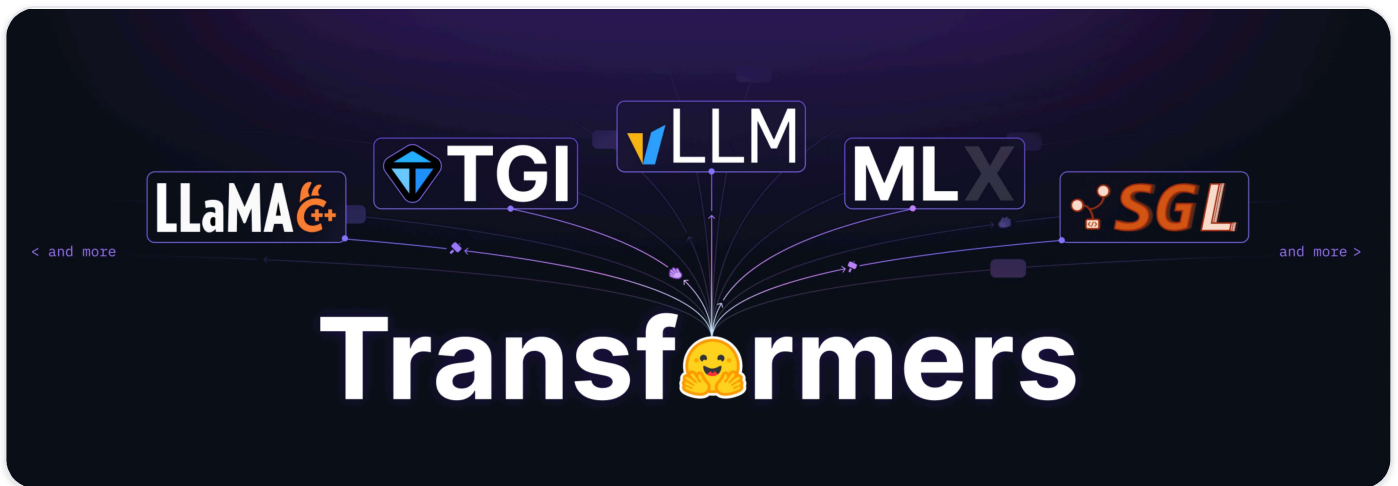
and get access to the augmented documentation experience

[Sign Up](#)

to get started



Transformers

[Copy page](#)

Transformers acts as the model-definition framework for state-of-the-art machine learning models in text, computer vision, audio, video, and multimodal models, for both inference and training.

It centralizes the model definition so that this definition is agreed upon across the ecosystem.

`transformers` is the pivot across frameworks: if a model definition is supported, it will be compatible with the majority of training frameworks (Axolotl, Unsloth, DeepSpeed, FSDP, PyTorch-Lightning, ...), inference engines (vLLM, SGLang, TGI, ...), and adjacent modeling libraries (llama.cpp, mlx, ...) which leverage the model definition from `transformers`.

[Ask HuggingChat](#)

We pledge to help support new state-of-the-art models and democratize their usage by having their model definition be simple, customizable, and efficient.

There are over 1M+ Transformers [model checkpoints](#) on the [Hugging Face Hub](#) you can use.

Explore the [Hub](#) today to find a model and use Transformers to help you get started right away.

Explore the [Models Timeline](#) to discover the latest text, vision, audio and multimodal model architectures in Transformers.

Features

Transformers provides everything you need for inference or training with state-of-the-art pretrained models. Some of the main features include:

- [Pipeline](#): Simple and optimized inference class for many machine learning tasks like text generation, image segmentation, automatic speech recognition, document question answering, and more.
- [Trainer](#): A comprehensive trainer that supports features such as mixed precision, torch.compile, and FlashAttention for training and distributed training for PyTorch models.
- [generate](#): Fast text generation with large language models (LLMs) and vision language models (VLMs), including support for streaming and multiple decoding strategies.

Design

Read our [Philosophy](#) to learn more about Transformers' design principles.

Transformers is designed for developers and machine learning engineers and researchers. Its main design principles are:

1. Fast and easy to use: Every model is implemented from only three main classes (configuration, model, and preprocessor) and can be quickly used with the [Trainer](#).



2. Pretrained models: Reduce your carbon footprint, compute cost and time by using a pretrained model instead of training an entirely new one. Each pretrained model is reproduced as closely as possible to the original model and offers state-of-the-art performance.



Learn

If you're new to Transformers or want to learn more about transformer models, we recommend starting with the [LLM course](#). This comprehensive course covers everything from the fundamentals of how transformer models work to practical applications across various tasks. You'll learn the complete workflow, from curating high-quality datasets to fine-tuning large language models and implementing reasoning capabilities. The course contains both theoretical and hands-on exercises to build a solid foundational knowledge of transformer models as you learn.

</> [Update](#) on GitHub

Installation →

